

Reviewer Pack — Hereditary Discrepancy of Clause-Variable Incidence Lower-Bo...

Entry 15736a2ab8cf · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-28 17:25:11 UTC

Hereditary Discrepancy of Clause-Variable Incidence Lower-Bounds Tree-Resolution Leaves

Entry ID: 15736a2ab8cf **Verdict:** INCONCLUSIVE **Recorded:** 2026-04-28 17:25:11 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial discrepancy theory: hereditary discrepancy $\text{herdisc}(H)$ of a set system H , computed by the Lovasz-Spencer-Vesztergombi determinant lower bound $\text{detlb}(H) = \max_{\{k,S\}} |\det(M)|^{1/k}$ over $k \times k$ submatrices M of the incidence matrix of H restricted to a subset S of columns, in the Chazelle-Matousek-Bourgain-Kashin discrepancy tradition (Matousek ‘Geometric Discrepancy’ Ch.4; LSV 1986). Rarely deployed against proof complexity; distinct from the Beck-Fiala slack approach and from spectral / hypercontractive bounds. **Field B** (complexity object): Tree-like resolution refutation leaf count $L_T(F)$ of small unsatisfiable 3-CNF formulas F , equivalently lex-DPLL search-tree leaf count under unit propagation; the canonical proof-complexity proxy for refutation hardness.

Statement:

Let F be an unsatisfiable 3-CNF formula on $n \leq 20$ variables and m clauses, and let $H(F)$ be the clause-variable incidence hypergraph (clauses as hyperedges over n variables, ignoring sign). Let $\text{detlb}(F) = \max_{2 \leq k \leq \min(8,m,n)} \max_{S \text{ of size } k} |\det(M_S)|^{1/k}$ where M_S is the $k \times k$ submatrix of largest absolute determinant among k rows of the ± 1 signed incidence matrix restricted to S (computed by exhaustive enumeration over the $C(m,k)$ row choices, capped at 5000 random samples when $C(m,k)$ exceeds it). Conjecture: $\log_2(L_T(F) + 1) \geq c \cdot \text{detlb}(F) \cdot \sqrt{n}$ for an absolute constant $c \geq 0.20$, with a single counterexample (any F where the inequality fails) refuting the bound.

Rationale (proposer’s reasoning):

The Lovasz-Spencer-Vesztergombi determinant lower bound is the strongest constructive lower bound on hereditary discrepancy and detects ‘rich’ submatrix structure that, in the Chazelle-Matousek-Bourgain-Kashin tradition, certifies dispersion-style obstructions; tree-resolution leaves on a CNF correspond to coverings of the input cube by partial-assignment cylinders constrained by clause incidence, so a high-determinant signed submatrix should force any DPLL branching tree to expose at least k linearly independent sign patterns and hence $\sim 2^k$ leaves. $\sqrt{n}$ calibration matches the typical Komlos / Spencer-style scaling for ± 1 matrices and avoids the natural-proofs barrier because detlb is sensitive to clause signs (not just the underlying hypergraph) and is not closed under generic monotone shifts. The bridge is rarely traversed: Beck-Fiala slack has been

tried (blacklisted) but determinant lower bounds and Chazelle’s hereditary-discrepancy sampler have not been linked to DPLL leaves.

Taxonomy category: DISPERSION_DISCREPANCY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9564295d698f1976

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across ≥ 200 unsatisfiable 3-CNFs (n in $\{8,10,12,14,16,18,20\}$, α in $\{4.0,4.5,5.0\}$) refuted by lex-DPLL within a 30M-leaf budget, compute $c_i = \log_2(L_T(F_i) + 1) / (\text{detlb}(F_i) \cdot \sqrt{n_i})$.
SUPPORTED iff $\min_i c_i \geq 0.20$; FALSIFIED iff any single instance yields $c_i < 0.20$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 11 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - hereditary discrepancy resolution proof complexity incidence matrix - determinant lower bound discrepancy DPLL tree resolution 3-CNF - Lovasz Spencer Vesztergombi discrepancy clause variable hypergraph refutation

Top relevant hits considered: - [<http://arxiv.org/abs/2212.02103v2>] A Study of Hypergraph Using Null Spaces of the Incidence Matrix and its Transpose - [<http://arxiv.org/abs/2111.02125v5>] Expected Complexity of Persistence Barcode Computation via Matrix Reduction - [<http://arxiv.org/abs/math/07024>] Topological invariants of piecewise hereditary algebras - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/0905.3806v1>] Limits of randomly grown graph sequences - [<http://arxiv.org/abs/1909.00234v1>] The spectrum of a class of uniform hypergraphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
import json
```

```

def generate_3cnf(n, m, alpha):
    clauses = set()
    while len(clauses) < m:
        clause = []
        for _ in range(3):
            var = random.randint(-n, n)
            if var not in clause and -var not in clause:
                clause.append(var)
        if len(clause) == 3:
            clauses.add(tuple(sorted(clause)))
    return list(clauses)

def lex_dp11(F, budget):
    n = max(abs(var) for var, _ in F)
    assignment = [None] * (n + 1)
    stack = []
    leaves = 0
    def dp11():
        nonlocal leaves
        if len(stack) == n:
            leaves += 1
            return True
        var = next((i for i in range(1, n + 1) if assignment[i] is None), None)
        if var is None:
            return False
        assignment[var] = True
        stack.append(var)
        if all(any(clause[i - 1] != assignment[abs(clause[i - 1])]) for i in clause) for clause in
            if dp11():
                return True
        assignment[var] = False
        stack.pop()
        assignment[-var] = True
        stack.append(-var)
        if all(any(clause[i - 1] != assignment[abs(clause[i - 1])]) for i in clause) for clause in
            if dp11():
                return True
        assignment[-var] = None
        stack.pop()
        return False
    dp11()
    return leaves

def signed_incidence_matrix(F, n):
    m = len(F)
    M = [[0] * n for _ in range(m)]
    for i, clause in enumerate(F):
        for var in clause:
            M[i][abs(var) - 1] = 1 if var > 0 else -1
    return M

def max_row_norm(M):
    return max(sum(abs(x) for x in row) for row in M)

def submatrix_determinant(M, S):
    m = len(S)

```

```

n = len(S[0])
M_S = [[M[i][j] for j in S[0]] for i in range(m)]
det = 1
for i in range(n):
    max_row = -1
    max_norm = -1
    for r in range(i, m):
        norm = sum(abs(M_S[r][c]) for c in range(n))
        if norm > max_norm:
            max_row = r
            max_norm = norm
    M_S[i], M_S[max_row] = M_S[max_row], M_S[i]
    det *= M_S[i][i]
    if det == 0:
        return 0
    for r in range(i + 1, m):
        factor = M_S[r][i] / M_S[i][i]
        for c in range(n):
            M_S[r][c] -= factor * M_S[i][c]
return det

def detlb(F, n):
    m = len(F)
    k_max = min(8, m, n)
    samples = 5000 if math.comb(m, k_max) > 5000 else math.comb(m, k_max)
    max_det = -1
    for _ in range(samples):
        S = random.sample(range(n), k_max)
        det = submatrix_determinant(F, [S])
        if det > max_det:
            max_det = det
    return max_det ** (1 / k_max)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16, 18, 20]
    alpha_values = [4.0, 4.5, 5.0]
    c_emp = float('inf')
    counterexample = ""
    for n in n_values:
        for alpha in alpha_values:
            F = generate_3cnf(n, int(alpha * n * (n - 1) / 6), alpha)
            leaves = lex_dp11(F, 30e6)
            if leaves == 0:
                continue
            detlb_val = detlb(F, n)
            c = math.log2(leaves + 1) / (detlb_val * math.sqrt(n))
            if c < c_emp:
                c_emp = c
            counterexample = f"n={n}, alpha={alpha}"
    return {
        "metric_name": "c_emp",
        "metric_value": c_emp,
        "instances_tested": len(n_values) * len(alpha_values),
        "conjecture_holds": c_emp >= 0.20,
        "counterexample": counterexample
    }

```

```

    }

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) or [11, 23, 37, 53, 71]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {json.dumps(result)}")
        results.append(result)

    mean_c_emp = sum(r["metric_value"] for r in results) / len(results)
    std_c_emp = math.sqrt(sum((r["metric_value"] - mean_c_emp) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_c_emp} std={std_c_emp} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='{result['counterexample']}'\n" first_failing_seed=first_failing_seed)
    else:
        print("RESULT: INCONCLUSIVE insufficient data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ae6da60.py", line 136, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ae6da60.py", line 116, in run_trial
    leaves = lex_dp11(F, 30e6)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ae6da60.py", line 31, in lex_dp11
    n = max(abs(var) for var, _ in F)
          ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2ae6da60.py", line 31, in <genexpr>
    n = max(abs(var) for var, _ in F)
          ~~~~~^
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a `ValueError` during clause unpacking in `lex_dp11` (clauses are 3-tuples, not 2-tuples), so no instances were evaluated and neither the support nor falsification condition can be assessed. | next: Fix the clause iteration in `lex_dp11` (unpack 3-literal clauses correctly, e.g., compute `n` via `max(abs(lit) for clause in F for lit in clause)`) and rerun the 200-instance battery.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	26250
2	preregistration	claude_max	opus	0	0	5976
3	novelty	claude_max	opus	0	0	2848
4	novelty	claude_max	opus	0	0	7411
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15131
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15528
7	judge	claude_max	opus	0	0	5390

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 78535 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/15736a2ab8cf.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/15736a2ab8cf.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/15736a2ab8cf.tar.gz` (if generated)